

《GC 停顿时间过长告警处理方案》

【告警名称】

- 告警名: HighGCPauseTime
- 告警级别: 严重
- 触发条件: 单次GC停顿时间持续超过1秒, 或Full GC频率超过每分钟1次

【问题描述】

当JVM出现GC停顿时间过长时, 可能导致:

- 应用暂停响应, 接口耗时出现明显毛刺
- 频繁Full GC会占用大量CPU资源, 进一步拖慢整体性能
- 严重时可能触发上游调用超时, 引发连锁反应

【排查步骤】

* 步骤1: 获取当前时间

工具: get_current_time

目的: 确定告警发生的时间范围, 用于后续日志查询

* 步骤2: 查询GC日志

工具: query_logs

参数要求:

- 地域: ap-guangzhou
- 日志主题: gc-logs
- 时间范围: 最近30分钟
- 查询条件: pause_time_ms:>1000

* 步骤3: 分析GC类型与频率

查看日志中GC事件明细, 重点关注:

- 是Young GC还是Full GC占主导
- GC后老年代内存回收效果如何
- 是否存在GC频率随时间加快的趋势

* 步骤4: 查询应用日志

工具: query_logs

参数要求:

- 地域: ap-guangzhou
- 日志主题: application-logs

- 时间范围：与GC停顿时间窗口一致
- 查询条件：level:ERROR OR level:WARN

【常见原因分析】

* 原因1：老年代内存持续增长（内存泄漏）

特征：

- Full GC后老年代占用率仍然很高，回收效果差
- 老年代使用率随时间持续攀升
- 与内存使用率过高告警场景高度相关

处理方案：

1. 生成堆转储文件分析老年代对象构成
2. 定位无法被回收的对象及其引用链
3. 修复内存泄漏问题后重新观察GC曲线

* 原因2：堆内存配置不合理

特征：

- 堆内存总量偏小，业务正常运行也频繁触发GC
- 新生代与老年代比例配置不当
- 调大堆内存或调整分区比例后GC频率明显下降

处理方案：

1. 结合业务实际内存需求评估堆大小配置
2. 调整新生代/老年代比例，减少对象过早晋升
3. 灰度调整JVM参数后观察GC表现

* 原因3：大对象或短生命周期对象创建过多

特征：

- Young GC频率异常高
- 存在批量创建大对象或临时对象的代码逻辑
- 对象晋升到老年代的速度过快

处理方案：

1. 排查是否存在不必要的大对象创建
2. 优化对象复用，减少临时对象产生
3. 评估是否可以使用对象池等技术降低分配频率

【紧急处理措施】

*** 立即操作（5分钟内）**

1. 重启：如老年代已严重碎片化，考虑重启服务
2. 扩容：临时增加实例数量分担流量压力
3. 降级：对非核心功能临时降级，减少内存压力

*** 短期措施（30分钟内）**

1. 持续观察GC频率与停顿时间变化
2. 生成堆转储文件保留现场
3. 排查具体的内存增长来源

*** 长期优化**

1. 建立GC停顿时间与频率的持续监控
2. 完善JVM参数配置的定期评估机制
3. 针对高频分配场景做专项性能优化

【验证步骤】

1. 确认单次GC停顿时间回落到200ms以下
2. 检查Full GC频率恢复到正常水平
3. 观察1小时确保GC表现保持稳定

【相关告警】

- HighMemoryUsage：内存使用率过高
- SlowResponse：响应时间过长

【联系方式】

- 运维团队：ops-team@company.com
- 开发团队：dev-team@company.com